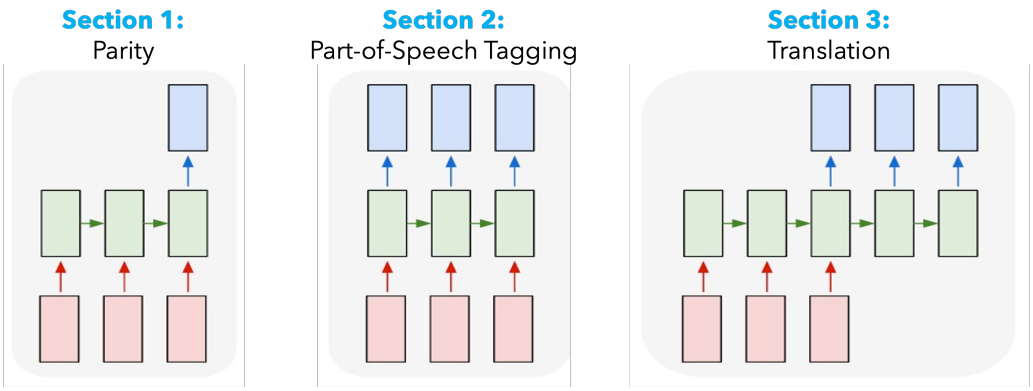
AI539 Natural Language Processing with Deep Learning – Homework 2

Processing Sequences with Recurrent Neural Networks

Overview and Objectives. In this assignment, we'll get some hands-on experience with training recurrent neural networks for sequence classification and conditional generation tasks – a simple parity problem, a part-of-speech tagging application, and machine translation with attention. As for general skills, we'll need to figure out how to handle batches of inputs with different lengths and use some tools from `wandb` to analyze our hyperparameter choices. These problems let us explore three common input-output schemes we discussed in lecture.



How to Submit. This assignment includes both coding questions and short response questions. Submit a zip file of your code and a PDF of your responses to Canvas. Do not include model checkpoints or training data.

Advice. Start early and be careful about shapes. For this assignment, we recommend getting on to the university high-performance cluster (HPC). See Canvas page for details.

1 A Simple Sequence Classification Problem: Parity

Let's probe LSTMs a bit by making one solve a fairly simple problem – determining if a binary sequence has an even or odd number of 1's. For context, there exists an LSTM with a 1-d hidden dimension that can perfectly solve this problem – it is a relatively fun puzzle to try to deduce it parameters manually. No guarantee of fun is implied. Professor not liable for any frustration.

1.1 Parity: Examining Generalization To Longer Sequences

To get started on this problem, I've provided a class in `parity/data/ParityData.py` that generates binary strings of certain lengths. During training, it provides random binary strings up to `max_length` and during testing it returns `samples=1000` binary strings of exactly `max_length`. The bulk of this class is shown below:

```python
class Parity(Dataset):

    def __init__(self, training=True, max_length=4, samples=1000):
      super().__init__()
      self.training = training
      self.max_length = max_length
      if self.training:
        self.data = torch.FloatTensor(list(itertools.product([0,1],
                                        repeat=max_length)))
      else:
        self.data = torch.randint(low=0, high=2, size=(samples,max_length))
        self.data = self.data.to(dtype=torch.float32)


    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        if self.training and self.max_length > 1:
           l = torch.randint(low=1, high=self.max_length, size=(1,1)).item()
           x = self.data[idx][:l]
        else:
           x = self.data[idx]

        y = x.sum() % 2  # Compute parity of the binary sequence
        return x,y
```

The file also provides a convenient `getParityDataloader` function shown below. However, our dataset contains strings of variable length so we need to pass a custom function to our data loader describing how to combine elements in a sampled batch together – in PyTorch, the `collate_fn` argument of the dataloader serves this purpose.

```python
def getParityDataloader(training=True, max_length=4, batch_size=1000):
   dataset = Parity(training, max_length)
   loader = DataLoader(dataset, batch_size=batch_size, shuffle=training,
                       collate_fn=dataset.pad_collate, drop_last=False)
   return loader
```

The code is currently set up to use a static method from the `Parity` dataset class to handle this – specifically, the `Parity.pad_collate` function which has the following form.

```python
class Parity(Dataset):
    . . .
    # Function to enable batch loader to stack binary strings of different
    lengths into one tensor padding them
    @staticmethod
    def pad_collate(batch):
        #TODO
        return xx, yy, x_lens
```

For a batch size of B, the input `batch` will be a list of B `x`,`y` tuples returned from the dataset's `__getitem__` function where each `x` will be a Tensor of arbitrary length and each `y` will be a Tensor of length 1. Note that if `num_workers` in the DataLoader is greater than 1, this list will be the result of multiple dataloader threads pooling their fetched data to form a batch, but this is transparent to us.

▶ TASK Q1 Batches from Random Length Strings [1 pt]. Implement the `Parity.pad_collate` function to produce the following outputs:

- `xx` – a $B \times T_{\max} \times 1$ FloatTensor representing the B `x` inputs of the batch where $T_{\max}$ is the length of the longest batch item. Any items shorter than $T_{\max}$ should be padded to this length by adding zeros. See the Pytorch pad_sequence function for a convenient way to do this.
- `yy` – a B LongTensor representing the B `y` labels of the batch.
- `x_lens` – a B LongTensor representing the length of each of the B `x` inputs of the batch.

Note that the requirement for `yy` and `x_lens` to be LongTensor types is to avoid errors when using the labels or lengths later on when computing cross entropy loss or when indexing into tensors based on lengths.

The `pairty.py` file in the root directory would be our main runner file that trains the LSTM model. With that behind us, the `pairty.py` file in the root directory implements a fairly standard training loop like we've seen before – except that the dataloader now has three outputs corresponding to `xx`, `yy`, and `x_lens` returned from our `pad_collate` function. The model being trained in an `ParityLSTM`; however, it has not yet been defined completely.

▶ TASK Q2 Parity Recurrent Neural Network Implementation [3 pt] Implement the unfinished `ParityLSTM` model class in `models/ParityLSTM.py`.

```
class ParityLSTM(nn.Module) :

    def __init__(self, hidden_dim=16):
        super().__init__()
        #TODO

    def forward(self, x, x_lens):
        #TODO
```

- **__init__**. For the model structure itself, there should be a one layer unidirectional LSTM with hidden dimension of `hidden_dim` and a single linear layer to map an output hidden state from the LSTM to two scalars corresponding to the score for predicting an even or odd parity.
- **forward**. During the forward pass, the input tensor should be processed by the LSTM to produce a $B \times T \times$ `hidden_dim` tensor of output hidden states through time. For each batch element i, the hidden state at time `x_lens[i]-1` should be collected – resulting in a $B \times$ `hidden_dim` tensor that can then be passed through the linear layer to produce a $B \times 2$ output that is then returned. PyTorch tensors use similar rules for indexing as numpy, so collecting the appropriate hidden states should be doable without doing any looping.

Once you get the `ParityLSTM` implemented, running `parity.py` should train your model on binary strings up to length 10 and log several things to wandb. This process should take only a couple of minutes on the GPU.

1.2 Utilities for Logging Experiments

Like most things in the world, deep learning models do not train in the first attempt and often require running extensive search on hyperparameters to find the best performing model. To be able to figure out where things are going wrong, logging desired values such as accuracy, loss, gradient norms, image/video visualizations etc. are of immense help. We've set up `wandb` logging for this parity problem and walk through the details here.

Weights&Biases. We can make use of experiment management tools to help us monitor our training and keep track of hyperpameters for us. In this section, we'll explore one such tool – Weights&Biases (aka `wandb`). *Why this tool?* Because all my graduate students swear by it and I haven't used a better tool so far. (Not a paid endorsement.)

Account Creation. Weights&Biases uses a web-interface for tracking experiment runs. This can either be a locally hosted server for personal projects or through their website. If you follow this link, you should be able to sign up for a free account. Once you've created your account, going to http://wandb.ai/authorize will let you copy your personal API key – *do not share this or leave it in your code.* The API key will be all we need to send information from our training scripts to the web interface.

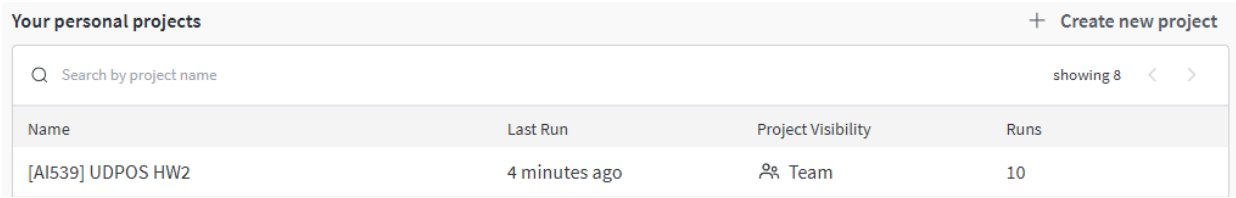
```
import wandb
wandb.login(key=YOUR_API_KEY_HERE) # Remember to remove this key before submitting.
```

Alternatively, calling just `wandb.login()` will produce a terminal prompt asking for your key. After providing it once, it will typically be cached for future runs – the `parity.py` code follows this route.

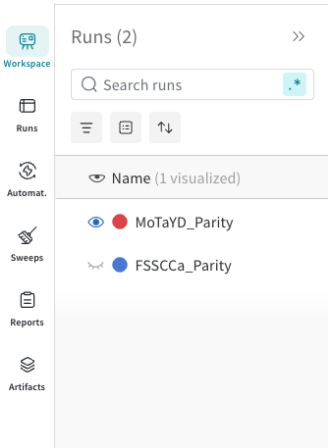
Before we train a new model, we can tell `wandb` what project it is a part of, what to name the current training run, and pass it our configuration dictionary to keep track of our hyperparameters for us. This will let us sort, filter, or group runs by these hyperparameters in the interface. An example call to `wandb.init` is shown below. The `generateRunName()` function is implemented in the skeleton code to generate a random string – e.g. `qX3Ax2_PARITY`.

```python
wandb.init(
    # Set the project where this run will be logged in the web interface
    project="[AI539] Parity HW2",
    # We pass a run name to identify this experiment
    name=generateRunName(),
    # Track hyperparameters and run metadata
    config=config)
```

On the website side, we will see a new project added to our profile that will store our runs.



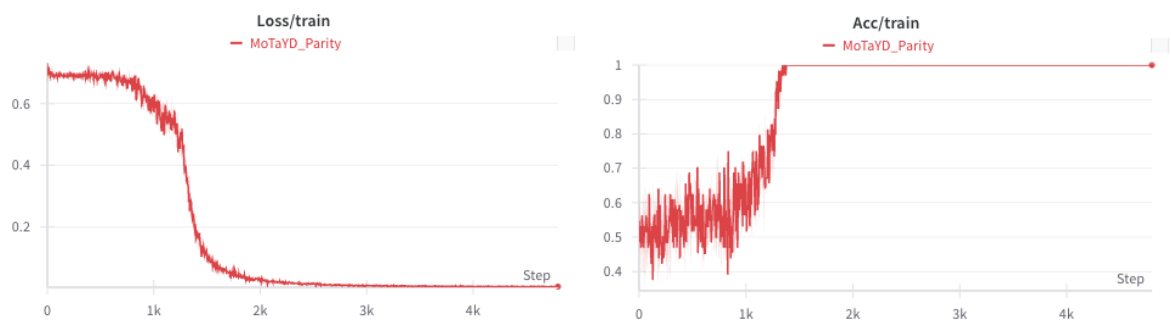
Clicking into that project, I can see information about prior runs I've logged to it.



Logging Values. After a Python script executes `login` and `init`, recording information about our training run to the web platform is quite easy. The primary command is `wandb.log` and it's most common arguments are a dictionary of named values to be logged and a `step` argument indicating how many batches of data have been processed in the training run so far. For instance, the line below logs the training loss and accuracy.

```python
loss = ... #Compute loss
acc = ... #Compute accuracy
wandb.log({"Loss/train": loss, "Acc/train": acc}, step=iteration)
```

If the above logging command is called at each batch of training, the web interface will aggregate the results over time and present the following plots inside the `[AI539] PARITY HW2` project for this run:

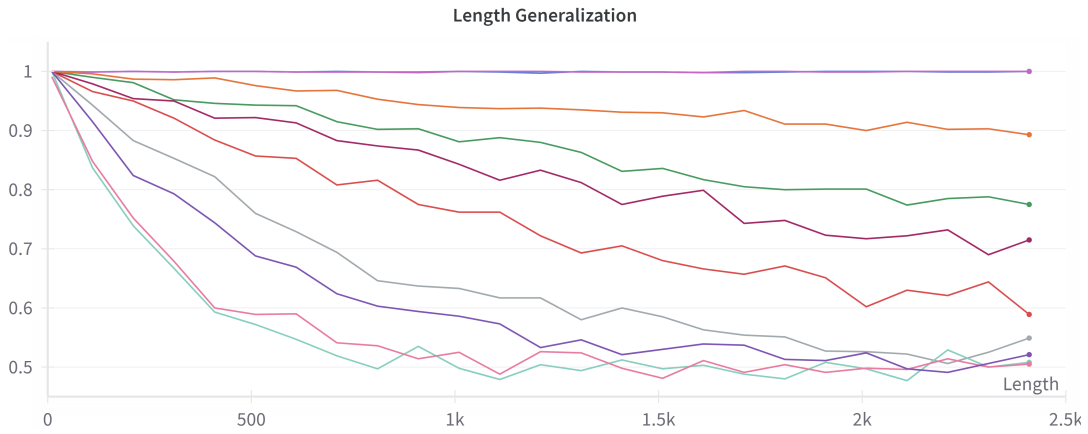


If we later log the test loss with the name "Loss/test", `wandb` will automatically group the two loss plots.

Wrapping Up. When we've finished our training run, we can notify `wandb` that it is over by calling `wandb.finish()`. This will ensure all logged data is successfully flushed to the website before our code exits.

1.3 Experimentation

After training, the code also executes the `runParityExperiment` function which evaluates the accuracy of your trained model on binary strings of increasing length up to 2500 – measuring generalization of your model to strings up to 250x longer than it saw during training! The results are sent to `wandb` as a plot and via two log variables `Text/acc` and `Test/length`. The plot below shows the results of this test for 10 different runs of the code. All runs achieve 100% training accuracy and are identical networks – only differing by the randomness of initialization and batch loading.



Some models (like the purple line at the top) generalize remarkably well, maintaining an accuracy near 100% across a wide range of lengths. Other models degrade as the input strings get longer. There is also substantial variance between different users – with some students reporting *all* runs behaving nearly identical and others replicating results like those shown above.

▶ `TASK Q3 Evaluating Generalization [1 pt].` With all of this randomness, its worth learning how to compare the effect of our hyperparameter choices over multiple runs using `wandb` by Grouping. First, let's generate some data from multiple training runs:

- Run the code 10 times with the default hyperparameters.
- Make a non-trivial change to a hyperparameter in `config` and then run the code 10 more times.

Run plot. To get a plot like the one above, you'll need to modify the x-axis of your `wandb` plot for `Test/acc` to be `Test/length` (see the cog wheel icon when mousing over the plot). Provide this plot in your report.

Grouped plot. By modifying the `Group` settings for this plot in `wandb`, group the runs by your changed hyperparameter and then provide the resulting plot. It should produce a mean and variance for both groups shown by solid and shaded regions. Be sure the plot's legend is enabled. Provide this plot in your report.

2 Part-of-Speech Tagging with Bidirectional GRUs

UDPOS Dataset. Now lets get to work on a real problem – part-of-speech tagging in English. This is a many-to-many sequence problem where each word in an input must be labeled with a corresponding part-of-speech tag. The goal of this section will be to train a bidirectional GRU model for this task. The dataset we will be using is the Universal Dependencies English Part-Of-Speech (UDPOS) benchmark which has these parts-of-speech annotated. The dataset has `train`, `dev`, and `test` sets made of approximately 12.5k, 2k, and 2k sentences each. These are largely news and email snippets. Example code visualizing an instance from the training set is shown below:

```
> x, y, lens = next(iter(train_loader))

> words = vocab.denumeralizeSentence(x[0,:lens[0]].flatten().numpy())
> labels = vocab.denumeralizeLabels(y[0,:lens[0]].flatten().numpy())

> visualizeSentenceWithTags(words, labels)

Token                POS Tag
---------------------------------
they                 PRON
are                  AUX
doing                VERB
it                   PRON
deliberately         ADV
.                    PUNCT
```

Unfortunately, words are not natively mathematical objects. We'll need to do some tokenization (breaking sentences down into discrete units called tokens) and numeralization (turning tokens into numbers) before we can have our models take them as input. The skeleton code provides a basic version of this functionality in `postagging/data/PoSData.py` via the `Vocab` class (which you may recognized from the previous assignment). Given a data split from UDPOS, the Vocab class finds all unique words and labels then builds dictionaries to quickly map between a word (or label) and its assigned numerical identifier. We expose this functionality through the numeralize / denumaralize functions demonstrated in the code snippet below.

```
> words = ['in', 'the', 'eastern', 'city', 'of', 'baqubah', ',']
> word_ids = vocab.numeralizeSentence(words)
[16, 12, 3007, 404, 18, 0, 11]

> words_back = vocab.denumeralizeSentence(word_ids)
['in', 'the', 'eastern', 'city', 'of', '<UNK>', ',']

> labels = ['ADP', 'DET', 'ADJ', 'NOUN', 'ADP', 'PROPN', 'PUNCT']
> label_ids = vocab.numeralizeLabels(labels)
[6, 5, 2, 3, 6, 0, 1]
```

Numeralizing an input converts it to corresponding word (or label) ID integers – e.g., the word 'in' being mapped to 16. These IDs are what our model will actually see rather than the words. Note that when we denumeralize `word_ids` above, the word `baqubah` is replaced with `<UNK>`. This happens because `baqubah` did not appear in our training set where the `vocab` was built, so it does not have a numerical ID. As such, it was mapped to a special ID of 0 when numeralized to indicate an unknown input. All such words will be mapped to this UNK ID and appear identical to our models. Further, we also include a special `<PAD>` token the in vocabulary to denote when batched inputs have been padded. Note that because this numeralization scheme is fixed, our model is only useful if we are encoding text with the same vocabulary that was used during training!

The skeleton code provides a convenient function to get dataloaders for all three splits and the training vocabulary. Note how the vocabulary from `train_data` is passed to the validation and test dataset constructors.

```
def getUDPOSDataloaders(batch_size=128):
   train_data = UDPOSDataset(split='train')
   val_data = UDPOSDataset(split='dev', vocab=train_data.vocab)
   test_data = UDPOSDataset(split='test', vocab=train_data.vocab)

   .
   .
   .

   return train_loader, val_loader, test_loader, train_data.vocab
```

And in the `UDPOSDataset` dataset class, we use the vocabulary to numeralize the inputs and labels into tensors of integers in the `__getitem__` function below:

```
class UDPOSDataset(Dataset):

    . . .

    def __getitem__(self, idx):
        x, y = self.data[idx]

        x = torch.LongTensor(self.vocab.numeralizeSentence(x))
        y = torch.LongTensor(self.vocab.numeralizeLabels(y))

        return x, y
```

Of course, both x and y can be different lengths for each element in our batch – we once again need a function for our dataloaders to handle batching our variable-length inputs! In this case though, we need to think a bit more about how to pad things appropriately.

▶ TASK Q4 Batches from Variable Lengths (Again) [0.5 pt]. Implement the pad_collate static method in the UDPOSDataset dataset class. Given a list of (x,y) pairs, return three tensors:

- xx – a $B \times T_{\max}$ LongTensor representing the B x inputs of the batch where $T_{\max}$ is the length of the longest batch item. Any items shorter than $T_{\max}$ should be padded to this length by adding 1's. Note that we are choosing 1's as this corresponds to the ID of the special "<PAD>" token in our vocabulary.
- yy – a $B \times T_{\max}$ LongTensor representing the B y labels of the batch.Any items shorter than $T_{\max}$ should be padded to this length by adding -1's. Note that we are choosing -1's here so we can identify these positions as not having labels (which have non-negative IDs).
- x_lens – a B LongTensor representing the length of each of the B batch sequences.

Bidirectional GRU Model. In this task, we need to predict a distribution over part-of-speech tags for each word in our batch. That is to say: for a $B \times T$ input of word IDs we need to produce a $B \times T \times |L|$ output where $|L|$ is the number of labels in our task. We will also need to configure our calculations for loss and accuracy accordingly. We will design a model that uses a bidirectional GRU as its primary sentence representation module.

We've consider our word representations as integer IDs so far; however, directly inputting these to our model is probably a bad idea. Two words having consecutive IDs (e.g., 16 and 17) is not semantically meaningful, but would look more similar to our model as input than words with distant IDs (e.g., 16 and 2045). To avoid this, it is useful to have each word represented by a learnable vector so that similarities and differences can be decided based on the data. For example, if the word "in" maps to ID 16, we would like some d_{embed}-dimensional vector e_{16} to represent it. These are commonly referred to as word embeddings or embedding vectors. For a $B \times T$ input of word IDs, we might collect the corresponding word embeddings for each word to produce a $B \times T \times d_{\text{embed}}$ tensor. PyTorch offers a useful nn.Embedding layer that does this efficiently for tensors of integer IDs.

▶ TASK Q5 Implement PoSGRU [4 pt]. In models/PoSGRU.py complete the PoSGRU class shown below by implementing the following:

```
class PoSGRU(nn.Module) :

    def __init__(self, vocab_size=1000, embed_dim=16, hidden_dim=16,
    num_layers=2, output_dim=10, residual=True, embed_init=None):
        super().__init__()

    def forward(self, x):
```

- **__init__**. The model will consist of an Embedding layer of dimension embed_dim and size vocab_size, a Linear layer mapping from embed_dim to hidden_dim, num_layers bidirectional GRUs with hidden dimension of hidden_dim/2, and a classifier module consisting of a hidden_dim Linear layer, a GELU activation, and a output_dim Linear layer. Note that a bidrectional GRU will output a concatenation of the forward and backward hidden states, such that setting its dimensionality to hidden_dim/2 results in an hidden_dim dimensional output per time step. For the embedding layer, setting the appropriate padding_idx is good practice.
- **forward**. During the forward pass, the $B \times T$ input tensor of word IDs should be embedded to a tensor of word embeddings that are then mapped to hidden_dim with our linear layer. Then each GRU layer should be applied in sequence (with a residual connection after each layer if residual is True). The result of this process should be a $B \times T \times$ hidden_dim tensor that is then mapped to $B \times T \times$ output_dim by the classifier and then returned.

Structure of PyTorch Training Code. Generally speaking, a standard training script in PyTorch will consist of the following steps: (1) construction of a dataset and dataloader to provide batches of training data, (2) definition of a model to be trained, (3) registration of model parameters to an optimizer, and (4) a training loop that takes batches of data, computes the model's loss on the batch, calculates gradients via backpropagation, and updates parameters via the optimizer. Note that we have a variable number of outputs and labels for each element of our batch! This requires some extra work to get right with the standard CrossEntropyLoss function.

▶ TASK Q6 API Hunt! Setup optimizer and learning rate scheduler. [0.5 pt] To complete the `train` function in `train_pos.py` we'll need to implement an optimizer for our model parameters. Look for initializations and parameters of the optimizers at `torch.optim` pytorch documentation. Using a combination of these function, create the following optimizer setup – (a) an AdamW optimizer with `config['lr']` learning rate and `config['l2reg']` weight decay, (b) a sequential learning rate schedule to perform linear warmup (0.25x the learning rate at the start up to 1x the learning rate after 10% of the training epochs) followed by a cosine annealed learning rate for the remainder of the training epochs.

▶ TASK Q7 API Hunt! Implement Loss and Accuracy Calculations [1 pt] Write code to do the following in the training and evaluation loop inside `train_pos.py`:

Forward pass. Get the data from the `data_loader`, move it to the GPU and perform a forward pass through the `PoSGRU` model that you have implemented.
Compute cross entropy loss. Review the PyTorch documentation for CrossEntropyLoss to understand the required shapes to compute our loss in this problem. Further, we want to ignore any losses coming from padded tokens (which we denoted with -1 labels in `UDPOSDataset.pad_collate` earlier).

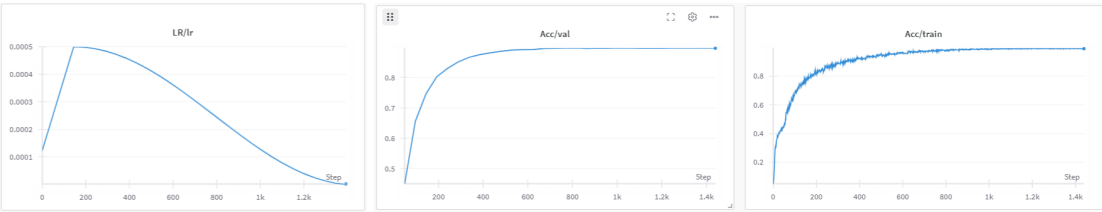
Compute accuracy. We also need to implement an accuracy calculation that likewise ignores padding and only considers our performance on actual input tokens. Implement one!

▶ TASK Q8 API Hunt! Implement Evaluation Function and Model Checkpointing [0.5 pt] In `train_pos.py`, complete the `evaluate` function to return the validation loss and accuracy. Then, use validation accuracy to perform model checkpointing. By checkpointing, we just mean saving model parameters each time you see a new highest validation accuracy. As our model depends on the specific vocabulary used during training, you'll also want to save that out. The easiest way is to save the entire vocabulary class from `train_loader.dataset.vocab` using the pickle package.

▶ TASK Q9 Integrate wandb logging [0.5 pt]. Use `wandb` to log the following:

- Learning rate as `LR/lr` once per epoch. You may need to look into the `get_last_lr` function for your learning rate scheduler to get the correct value.
- Training loss and accuracy per batch as `Loss/train` and `Acc/train` respectively.
- Validation loss and accuracy once per epoch as `Loss/val` and `Acc/val` respectively.

Once these are implemented, `train_pos.py` should be able to be run to train your model. You can expect GPU-based training times well under 5 minutes for this problem. `wandb` logging has been setup and you should see training accuracies near 100% and validation accuracies over 85% for default parameters. Sample plots for learning rate and accuracy are shown below:



2.1 Experimentation

▶ TASK Q10 Justifying our GRU [0.5 pt] When `layers` is set to 0 in the `config` in `train_pos.py`, there are no GRUs at all in our model. Instead, each word is embedded and processed by a few linear layers independently. Using your new knowledge about `Grouping` in `wandb`, run multiple trials and use the graphed means and variance to argue for the choice of a GRU here over this word-independent `layers=0` baseline. Run at least 10 random seeds for each setting. Report the resulting plot to justify your argument.

▶ TASK Q11 Respecting Residual Connections [0.5 pt] The `config` also allows us to enable or disable the residual connections. We can examine its effect on training and the gradients in our model.
To measure the impact of residuals on gradient flow, we can measure the norm of the gradient matrix for the earliest layer of our network – the embedding layer `model.embed`. After the backwards pass, these can be accessed at `model.embed.weight.grad`. Modify the train loop to also log the norm of the embedding gradients.Adjust the GRU `layers` to 25 and run training. Then disable the residual connections (`residual=False`) and rerun the experiment.Report `wandb` plots of the gradient norm and training loss and comment on your observations.

As this is a simple task without too much data, a 25-layer model is likely to overfit compared to our default 2-layer model, so be sure to renable residuals and revert to 2 layers for the following tasks.

Using GloVe vectors: Instead of training the word embeddings from scratch, let's use GloVe that we learned about in Week 1 as an initialization. We will make use of the `gensim` library and you will have to `pip install gensim`.

```
import  gensim.downloader
glove_wv = gensim.downloader.load('glove-wiki-gigaword-100')
```

Now, you can use the downloaded word vectors to query the learned embedding:

```
>> glove_wv['attention'] # look at the GloVe embedding for the word "attention"
array([ -3.3414e-01,  4.6667e-01, ..., 6.7652e-01], dtype=float32)
>> glove_wv['attention'].shape # look at the shape
(100,)
>> glove_wv['GRU'] # Let's see if we can get the embedding for "GRU"
KeyError: "Key 'GRU' not present"
```

As you can see there will be some words that do not exist in the vocabulary of the corpus on which the GloVe embeddings were trained on. So how can we ensure that there exists a representation for them? You can randomly initialize them (normally distributed with mean 0 and variance 1). Note that even though you initialize the world vectors with GloVe embeddings, you still would like to learn (in machine learning lingo – *finetune*) the word representation.

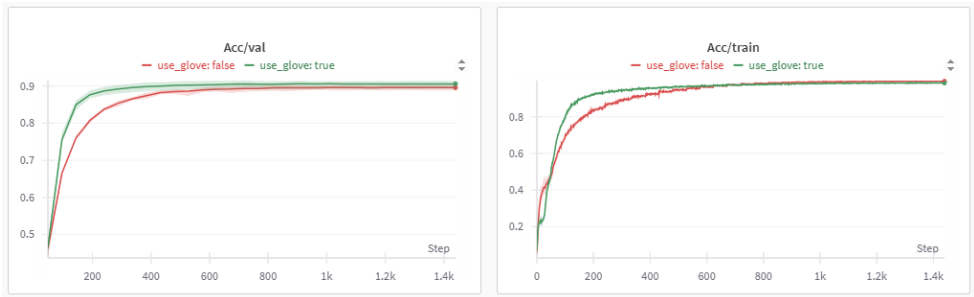
▶ TASK Q12 Use GloVe word vector initializations. [2 pt] To have our model make use of pre-trained GLoVe embeddings, you'll need to:

- Modify `main()` in the `train_pos.py` to build a `vocab_size` $\times$ 100 dimension tensor initialized by GLoVe vectors for our vocabulary words. Pass this as the `embed_init` argument to the `PoSGru` constructor. You may use the above boilerplate code for loading the GloVe embeddings.
- Modify the PoSGRU `__init__` function to initialize the `nn.Embedding` layer with `embed_init` if it is not `None`. The `nn.Embedding.from_pretrained` function can be useful for this, but you'll want to ensure the parameters are not frozen and that no normalization of the vectors occurs – check the API for appropriate arguments.

After you've made these modifications, train the model and answer:

1. How many words did you encounter that did not have GloVe embeddings? List 2-3 of these words.
2. Report the loss and accuracy plots from *wandb* comparing the difference in your model with and without GLoVe embeddings. Describe your observations.

In our experiments with a 2-layer GRU and otherwise default hyperparameters, we see initializing from GLoVe vectors leads to faster improvement on the validation set and some gains in accuracy over the random initialization. The plot below is grouped by GLoVe vector usage and consists of 10 split between the two settings.



2.2 Inference

Finally, let's make use of your trained model on some example sentences. To do so, we'll need to build a driver program that loads your trained model, applies the same tokenization/numeralization scheme as from model training, and then runs inference. *Hint: Be careful with capitalization and whitespace here!*

▶ TASK Q13 Build driver script and run inference. [5 pt] Implement a script named `tag.py` that prompts the user for an input sentence when run and returns the part-of-speech tags predicted by your trained model. This will require you to tokenize/numeralize the sentence, pass it through your network, and then print the denumeralized results. Run your script on the following sentences and report the output:

1. The old man the boat
2. The complex houses married and single soldiers and their families
3. The man who hunts ducks out on weekends
4. Danielle's dog Bingbong blundered brilliantly at the Westminster Kennel Club Dog Show.

3 Attentive Sequence-to-Sequence Models for Machine Translation

Multi30k Dataset. Published in 2016, the Multi30k dataset supports machine translation between English and German (and later French). The dataset consists of roughly 30,000 image descriptions captured in both German and English by human translators. The skeleton code provides a convenient function to get dataloaders for all three splits and the training vocabulary for both German and English. Note again how the vocabularies from `train_data` are passed to the validation and test dataset constructors. The vocabularies themselves offer `tokenize` and `detokenize` functions for numeralization and denumeralization operations which will use language-specific tokenizers.

```
def getMulti30kDataloadersAndVocabs(batch_size=128):

  multi_train = Multi30kDatasetEnDe(split="train")

  multi_val = Multi30kDatasetEnDe(split="validation", vocab_en=multi_train.
    vocab_en, vocab_de=multi_train.vocab_de)

  multi_test = Multi30kDatasetEnDe(split="test",  vocab_en=multi_train.
    vocab_en, vocab_de=multi_train.vocab_de)
    .
    .
    .

  return train_loader, val_loader, test_loader, {"en":multi_train.vocab_en,
    "de":multi_train.vocab_de}
```

Similar to the previous section, the `pad_collate` function in `Multi30kDatasetEnDe` handles the variable length of input and output sequences – returning batches of padded, numeralized German (`xx_pad`) and English (`yy_pad`) sentences along with their lengths (`x_lens` and `y_lens`). Note that the dataset automatically prepends (appends) the English sentences with the `<SOS>` (`<EOS>`) so the decoder is trained to handle these tokens.

3.1 Banishing UNKs with Byte Pair Encoding Tokenization

Our tokenizer in the previous section was based on a fixed vocabulary which transforms rare words like `Westminster` in the final example of Q12 to UNK. As discussed in class, this limits the patterns our model can learn because all UNK words are mapped to the same representation. Byte-Pair Encoding (BPE) offers an alternative. Rather than committing to a fixed word-level vocabulary up front, BPE builds a vocabulary of *subword* units by iteratively merging the most frequent adjacent token pair in the training corpus. Starting from individual characters (so every word is always representable), this process produces a vocabulary that covers common words as single tokens while decomposing rare words into smaller, meaningful pieces. For instance, after enough merges the word `translate` might be a single token, while the rare proper noun `Bingbong` might be split into `B`, `ing`, `b`, `o`, `ng` – ugly, but *not* UNK.

Algorithm overview. The BPE tokenizer relies on token co-occurence frequencies so must be trained on a text corpus. The training procedure iterates the following steps until either some maximum number of merges have been performed or no token pair appears more than once:

1. Count the frequency of every adjacent token pair across the corpus.
2. Select the most frequent pair (a, b) and create a new token ab by concatenating the strings they represent.
3. Replace every occurrence of the pair (a, b) in the corpus with the new token ab.

To tokenize new text, we start from individual characters and apply each merge rule in the order it was learned.

Example. Let's take an example to make this algorithm clear. Consider the following training corpus:

```
"low low lower lower newer new newest newest"
```

and start our vocabulary off with tokens `l, o, w, n, e, r, s, t` and space (denoted "_" here for visual clarity). Such that our vocabulary with corresponding numeralization would be:

Token	l	o	w	n	e	r	s	t	_
ID	0	1	2	3	4	5	6	7	8

In practice, vocabularies are typically initialized with an exhaustive set of Unicode characters, but for our demonstration this smaller set makes things simpler. Our initial tokenization and numeralization of this corpus would be:

```
l o w _ l o w _ l o w e r _ l o w e r _ n e w _ n e w e r _ n e w e s t _ n e w e s t
0 1 2 8 0 1 2 8 0 1 2 4 5 8 0 1 2 4 5 8 3 4 2 8 3 4 2 4 5 8 3 4 2 4 6 7 8 3 4 2 4 6 7
```

Counting all pairs of sequential tokens, we find the following pairwise frequencies:

w,e	l,o	o,w	_,n	n,e	e,w	_,l	e,r	r,_	e,s	s,t	w,_	t,_
5	4	4	4	4	4	3	3	3	2	2	1	1

We select the most frequent and add `we` as a new token in our vocabulary with token ID 9. Then we would merge all adjacent `w,e` pairs in the corpus to get a new tokenization/numeralization

```
l o w _ l o w _ l o we r _ l o we r _ n e w _ n e we r _ n e we s t _ n e we s t
0 1 2 8 0 1 2 8 0 1 9  5 8 0 1 9  5 8 3 4 2 8 3 4 9  5 8 3 4 9  6 7 8 3 4 9  6 7
```

where each `we` token is just represented by a single ID. This completes one iteration of the algorithm. At the start of the next round, the co-occurrences are recomputed.

l,o	n,e	_,n	w,_	we, r	r,_	_,l	e,we	we,r	o,w	o,we	we,s	s,t	e,w	t _
4	4	4	3	3	3	3	3	3	2	2	2	2	1	1

Note that because `w,e` was replaced with `we` the counts for `e,w` pairs dropped to 4 to 1 as only the e,w in 'new' persists. This highlights why we need to recompute relevant pair frequencies at each step.

Word-level BPE with Pre-tokenization. If an entire text corpus is concatenated and then the above algorithm is run for enough merges, we can learn interesting tokens like `San Francisco` where the merges have crossed word boundaries to represent a multi-word concept with a single token! Unfortunately, that version of this algorithm is a bit costly for us to run on large corpuses without significant parallelization – requiring a full pass of the corpus for each merge. To make things cheaper, we can restrict ourselves to word boundaries in a step known as pre-tokenization. Specifically we will use the following regular expression to divide our corpus

```
"'s|'t|'re|'ve|'m|'ll|'d| ?[A-Za-z]+| ?\d+| ?[Â-Za-z\d\s]+|\s+(?!\S)|\s+"
```

which breaks posessives and contractions first (`'s|'t|'re|'ve|'m|'ll|'d`), matches words with or without a preceding space (`?[A-Za-z]+`), numbers with our without a preceding space (`?\d+`), non-alphanumeric strings like punctuation with or without a preceding space (`?[Â-Za-z\d\s`), and finally clears any trailing whitespaces (`s+(?!\S)|\s+`). Applying this to our example corpus, we get

low, _low, _lower, _lower, _newer, _new, _newest, _newest

With our corpus divided into words, we can now reason about multiple instances of words in our corpus all at once by building a word-frequency table from the corpus.

Word (as token tuple)	Count
(l o w)	1
(_ l o w)	1
(_ l o w e r)	2
(_ n e w)	1
(_ n e w e r)	1
(_ n e w e s t)	2

Computing Pair Frequencies. To compute frequencies for some token pair a,b, we can iterate through this list of words and add counts equal to the number of times a,b occurs in the word $\times$ the count of the word. For example, the pair w,e occurs once in (_ l o w e r), once in (_ n e w e r), and once in (_ n e w e s t) for a total of $1 \times 2 + 1 \times 1 + 1 \times 2 = 5$. Note that if our corpus included a word like (_ n e w e w e) that occurred 4 times in our corpus, it would contribute $2 \times 4 = 8$ to the pair frequency of w,e.

Merging. Merging works similarly to our previous example, except now operates per word – e.g., adding a we token and merging yields the updated table.

Word (as token tuple)	Count
(l o w)	1
(_ l o w)	1
(_ l o we r)	2
(_ n e w)	1
(_ n e we r)	1
(_ n e we s t)	2

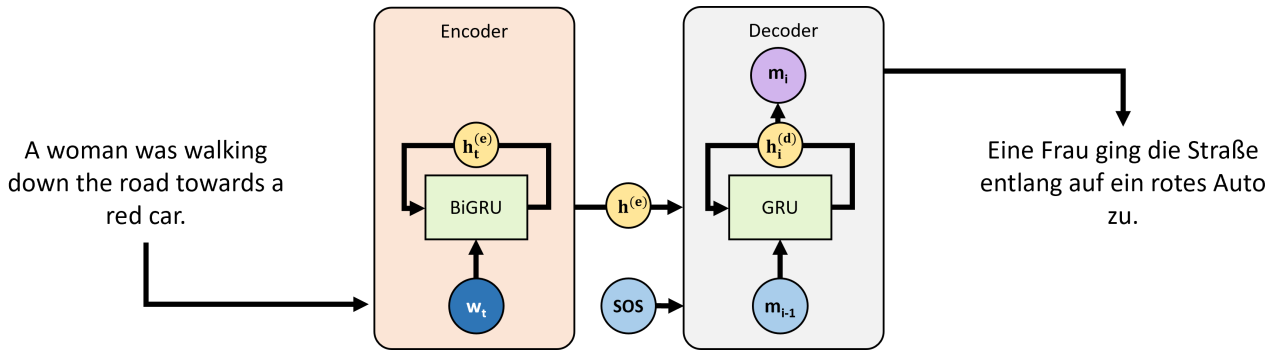
This reduces our per-round compute to linear in the number of unique words rather than the length of our corpus.

▶ TASK Q14 Finish BPE Implementation [2pt]. In translation/data/BPETokenizer.py, complete the BPE implementation by completing the functions shown below to implement the above algorithm.

```
@staticmethod
def _pair_freqs(word_freqs: dict[tuple[int,...], int]) -> dict[tuple[int, int], int]:
    """
    Count every adjacent id-pair across all word-types, weighted by
    word frequency.  O(V * avg_word_len) where V = unique words.
    """
    # TODO: Return a dictionary with token id tuples (a,b) as keys and their
    occurence count as value.
    return freqs


@staticmethod
def _apply_merge(ids: list[int], a: int, b: int, new_id: int) -> list[int]:
    """Replace every (a, b) occurrence in *ids* with *new_id*."""

    #TODO: Return a list of ids where any instance of a followed by b is replaced
    with new_id

    return out
```

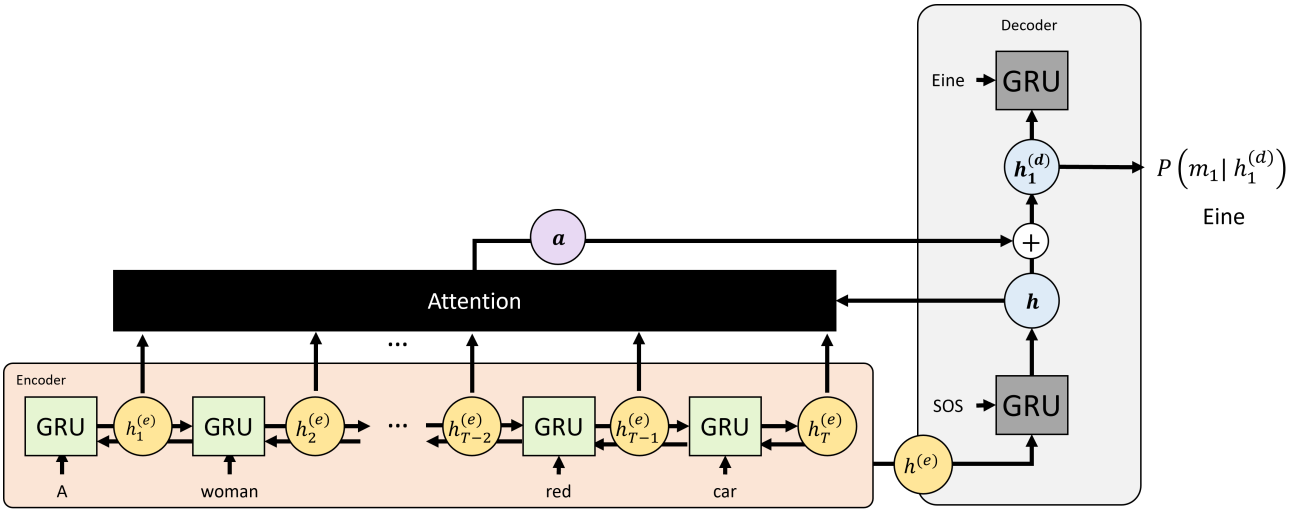
As with most educational materials, designing BPE from scratch is a great exercise for understanding how it works, but doesn't result in a production-ready implementation. In your own project, open-source codebases like tiktoken or Huggingface will offer stronger performance and better edge-case handling.

3.2 Sequence-to-Sequence Models



We are going to be considering a class of sequence-to-sequence models based on GRUs for this machine translation task – also known as encoder-decoder models. The simplest of these models (shown above) consists of an *encoder* bidirectional GRU that takes in a numeralized German sentence and a *decoder* unidirectional GRU that produces the corresponding English. During training, the decoder's hidden state is initialized by a representation from the encoder and then the decoder's outputs are supervised to assign high probability to the next word in the training English sentence via cross entropy loss.

As discussed in class, this basic structure relies on the encoder's output to capture everything relevant about the input and then the decoder's ability to retain that information over multiple timesteps. This can be difficult so adding an attention mechanisms that can "*look back*" at different parts of the input representation can potentially improve performance. We will consider different forms for this attention mechanism, but a general outline is shown below.



In this setting, the hidden state output h of our decoder GRU is used as a query to some attention process and the bidirectional hidden states of the encoder GRU are used as candidates. The resulting vector from attention is then added to h to make the attention-augmented state $h_1^{(d)}$ which is used to predict a distribution over the next word and is passed to the next time step of the decoder GRU.

Encoder. Let's formalize this a bit mathematically to be more concrete about what is happening. Consider a training example consisting of a source language sentence $w_1, ..., w_T$ and a target language sentence $m_1, ..., m_L$. Let $\mathbf{w}_t$ and $\mathbf{m}_i$ be one-hot word vectors such that

$$\mathbf{z_t} = \text{Dropout}(\mathbf{W}_e \mathbf{w}_t) \qquad \text{(Word Embedding)} \qquad (1)$$

represents a word embedding with dropout applied. In PyTorch we will use word IDs and `nn.Embedding` layers rather than performing this sparse matrix-vector product to reduce computation. Given the word embeddings $z_1, ..., z_T$ for the source sentence, a bidirectional GRU computes hidden representations.

$$\overrightarrow{\mathbf{h}}_t^{(e)} = \overrightarrow{\text{GRU}}\left(\mathbf{z}_t, \overrightarrow{\mathbf{h}}_{t-1}^{(e)}\right) \qquad \text{(Forward GRU)} \qquad (2)$$

$$\overleftarrow{\mathbf{h}}_t^{(e)} = \overleftarrow{\text{GRU}}\left(\mathbf{z}_t, \overleftarrow{\mathbf{h}}_{t+1}^{(e)}\right) \qquad \text{(Backward GRU)} \qquad (3)$$

$$\mathbf{h}_t^{(e)} = \left[\overrightarrow{\mathbf{h}}_t^{(e)}, \overleftarrow{\mathbf{h}}_t^{(e)}\right] \forall t \qquad \text{(Word Representations)} \qquad (4)$$

where $[\cdot,\cdot]$ denotes concatenation. While we write the forward and backward networks separately, PyTorch implements this as a single API call. To produce a sentence-level representation to initialize the decoder, we can take the final hidden states of the forward and backward GRUs:

$$\mathbf{h}^{(\mathbf{e})} = \left[\overrightarrow{\mathbf{h}}_T, \overleftarrow{\mathbf{h}}_0\right] \qquad \text{(Sentence Representation)} \tag{5}$$

▶ TASK Q15 Implement Encoder [3pt]. In `models/Seq2SeqTranslator.py`, complete the implementation of the `BidirectionalEncoder` class by completing the functions shown below to implement the equations discussed in the previous section.

```
class BidirectionalEncoder(nn.Module):
    def __init__(self, src_vocab_len, emb_dim, enc_hid_dim, dropout=0.5):
        super().__init__()

        #TODO

    def forward(self, src, src_lens):

        #TODO

        return word_representations, sentence_rep
```

For a given source sequence of word IDs `src`, the model should embed each word ID to a `emb_dim` word embedding, apply Dropout with `dropout` probability to these embedding, then pass them through a 1-layer bidirectional GRU with hidden dimension of size `enc_hid_dim`. It should return the bidirectional hidden states for each word as a $B \times T \times 2*$`enc_hid_dim` tensor `word_representations`. The `sentence_rep` should be a $B \times 2*$`enc_hid_dim` tensor produced by concatenating the forward GRU hidden state for the last word in each sentence with the zeroth backward GRU hidden state for each sentence. Note, you'll need to use indexing to pick out the final forward states based on `src_lens`. Further, separating the forward/backward representations will require studying the GRU documentation in PyTorch.

To map the sentence representation to match the decoder hidden state size, our sequence-to-sequence model can include a linear layer and initialize the decoder hidden state with

$$\mathbf{h}_0^{(d)} = \text{GELU}(\mathbf{W}\mathbf{h}^{(e)} + \mathbf{b}) \qquad \text{(Initialize Decoder)} \tag{6}$$

For a standard seq-to-seq model without any attention mechanism, this would be all the information the decoder would get about the input sentence!

Decoder. Our decoder is a unidirectional GRU that performs an attention operation over the encoder word representations at each time step. For the ith word in the output sequence m_i, the decoder performs the following operations:

$$\mathbf{b_i} = \text{Dropout}(\mathbf{W}_d\mathbf{m}_i) \qquad \text{(Word Embedding)} \tag{7}$$

$$\mathbf{h} = \text{GRU}\left(\mathbf{b}_i, \mathbf{h}_{i-1}^{(d)}\right) \qquad \text{(Forward GRU)} \tag{8}$$

$$\mathbf{a}_i = \text{Attn}\left(\mathbf{h}, \mathbf{h}_1^{(e)}, ..., \mathbf{h}_T^{(e)}\right) \qquad \text{(Attention)} \tag{9}$$

$$\mathbf{h}_i^{(d)} = \mathbf{h} + \mathbf{a}_i \tag{10}$$

$$y = \mathbf{W}_d\mathbf{h}_i^{(d)} + \mathbf{b}_d \qquad \text{(Output Scores)} \tag{11}$$

where the probability of the next word can be computed from the output scores via a softmax as $P(m_{i+1} \mid m_{\leq i}, w_1, ..., w_T) =$ softmax(y). We will leave the specifics of the `Attn` function to the next section and for now assume it returns some vector $\mathbf{a}_i$ the same dimension as $\mathbf{h}$.

▶ TASK Q16 Implement Decoder [3pt]. In `models/Seq2SeqTranslator.py`, complete the implementation of the `Decoder` class by completing the functions shown below to implement the equations discussed in the previous section. Note that we are only considering the `forward` function for a single time step and will rely on the sequence-to-sequence model later to iterate through the target sentence.

```
class Decoder(nn.Module):
    def __init__(self, trg_vocab_len, emb_dim, dec_hid_dim, attention, dropout=0.5):
        super().__init__()

        self.attention = attention

        # TODO

    def forward(self, input_word, hidden, encoder_outputs):

        # TODO

        return hidden, out, alphas
```

For your implementation, assume that the provided `self.attention` supports the following operation:

```
attended_feature, alphas = self.attention(hidden, encoder_outputs)
```

where the `attended_feature` is the $B \times$ `dec_hid_dim` result of attention and `alpha` is the $B \times T$ tensor of attention values α over the input sequence. We will implement this in the next task.

The decoder itself will consist of word embeddings for the target language of dimension `emb_dim`, a unidirectional GRU with `dec_hid_dim` dimensional hidden states, and a classifier to predict word probabilities over the target vocabulary from the hidden state (`dec_hid_dim` dimensional linear layer, GELU, and a `trg_vocab_len` dimensional linear layer).

In the `forward` function, we will presume we are moving the decoder forward by only one time step. As such, the `input_word` is a B dimensional tensor of word IDs, `hidden` is a $B \times$ `dec_hid_dim` tensor containing the previous hidden states, and `encoder_outputs` is the $B \times T \times$ `2*enc_hid_dim` `word_representations` tensor returned from the `BidirectionalEncoder`. With these inputs, the `forward` function should embed the input words, apply dropout, and then step the GRU forward to produce an updated hidden state ($\mathbf{h}$). Then the attention function should be called to produce the attended feature and alphas. The updated hidden state and attended feature should be added to produce the new hidden state and the classifier should be applied to produce a $B \times$ `trg_vocab_len` tensor of word scores. The new hidden state, this output tensor, and the attention alphas should be returned.

Attention. Recall from the lecture the definition of a single-query scaled dot-product attention mechanism. Given a query $\mathbf{q} \in \mathbb{R}^{1\times d_{kq}}$, a set of candidates represented by keys $\mathbf{k}_1, ..., \mathbf{k}_T \in \mathbb{R}^{1\times d_{kq}}$ and values $\mathbf{v}_1, ..., \mathbf{v}_T \in \mathbb{R}^{1\times d_v}$, we compute the scaled dot-product attention (with the softmax explicitly written out) as:

$$\alpha_i = \frac{\exp\left(\mathbf{q}\mathbf{k}_i^T/\sqrt{d_{kq}}\right)}{\sum_{j=1}^{T}\exp\left(\mathbf{q}\mathbf{k}_j^T/\sqrt{d_{kq}}\right)} \quad \forall i \tag{12}$$

$$\mathbf{a} = \sum_{j=1}^{T}\alpha_j\mathbf{v}_j \tag{13}$$

where the α_i are collectively referred to as the attention distribution and $\mathbf{a}$ the attended feature.

▶ TASK Q17 Implement Attention Mechanism [3pt]. Complete the DotProductAttention class by completing the following functions in models/Seq2SeqTranslator.py to implement a batched version of the attention mechanism described above where the queries are generated from hidden state of our decoder and the keys / values from the encoder_outputs.

```
class DotProductAttention(nn.Module):

    def __init__(self, q_input_dim, cand_input_dim, v_dim, kq_dim=64):
        super().__init__()

        #TODO

    def forward(self, hidden, encoder_outputs):

        #TODO

        return attended_val, alphas
```

To transform the inputs to queries, keys, and values we will need three linear transformations: one to convert the q_input_dim dimensional hidden input to kq_dim, one to convert the cand_input_dim dimensional encoder_outputs input to kq_dim, and one to convert the cand_input_dim dimensional encoder_outputs input to v_dim.

In the forward function, the following operations should be performed

- The hidden input should be transformed to a $B \times$ kq_dim query tensor via a linear layer.
- The encoder_outputs input should be converted a to $B \times T \times$ kq_dim key tensor via a linear layer.
- The encoder_outputs input should be converted a to $B \times T \times$ v_dim value tensor via a linear layer.
- Use torch.bmm and appropriate permute operations to compute a $B \times 1 \times T$ output resulting from computing scaled dot products between each of the B query and key matrix pairs. Be sure to scale the dot products by the square root of kq_dim.
- Apply squeeze and softmax operations to produce an $B \times T$ alpha tensor from the scaled dot product tensor wherein each row sums to 1.
- Use torch.bmm and appropriate permute / unsqueeze operations to compute a $B \times$ v_dim attended feature tensor.

Return the attended feature and alpha tensors.

In addition to the attention mechanism you've implemented, the code provides two more – a Dummy attention that always returns a zero vector and a MeanPool attention that assigns equal attention weight to all inputs. The default hyperparameters use the DotProductAttention you implemented above.

Our full model. To put these modules together, we define a Seq2Seq model, the init of which can be seen below.

```
class Seq2Seq(nn.Module):
    def __init__(self, src_vocab_size, trg_vocab_size, embed_dim, enc_hidden_dim,
                 dec_hidden_dim, kq_dim, attention, dropout=0.5):
        super().__init__()

        self.trg_vocab_size = trg_vocab_size
        self.encoder = BidirectionalEncoder(src_vocab_size, embed_dim,
                                            enc_hidden_dim, dropout)
        self.enc2dec = nn.Sequential(nn.Linear(enc_hidden_dim*2,dec_hidden_dim), nn.GELU())

        if attention == "none":
            attn_model = Dummy(dec_hidden_dim)
        elif attention == "mean":
            attn_model = MeanPool(2*enc_hidden_dim, dec_hidden_dim)
        elif attention == "dotproduct":
            attn_model = DotProductAttention(dec_hidden_dim, 2*enc_hidden_dim,
                                             dec_hidden_dim, kq_dim)

        self.decoder = Decoder(trg_vocab_size, embed_dim, dec_hidden_dim,
                               attn_model, dropout)
```

It already constructs the encoder, a linear layer to convert the `sentence_rep` output from the encoder to the size of the decoder hidden state, and a decoder with the requested attention mechanism. We need to finish writing its forward pass and provide a utility function to decode outputs autoregressively at test time.

▶ TASK Q18 Implement Seq2Seq Model Forward [3pt]. Complete the Seq2Seq model by implementing the following function in `models/Seq2SeqTranslator.py`. This function will be used during training to compute predicted scores for each next word in a training example.

```
class Seq2Seq(nn.Module):
    . . .
    def forward(self, src, trg, src_lens):
        #tensor to store decoder output scores
        outputs = torch.zeros(trg.shape[0], trg.shape[1],
                              self.trg_vocab_size).to(src.device)

        #TODO

        return outputs
```

In the `forward` function, the source sentence `src` should be processed by the `encoder` to yield the `word_representations` and `sentence_rep` tensors. The `enc2dec` layer should be used to initialize the hidden states of the `decoder` with transformed `sentence_rep`. Then, for each time step t in the target sentence `trg` of length L,

- The decoder model should be applied to step forward one step – taking the ground truth `trg` words at time t as input and updating the hidden state and producing an $B \times$ `trg_vocab_size` output tensor of next-word scores for each vocabulary word.
- These predicted scores should be stored in the $B \times L \times$ `trg_vocab_size` `outputs` tensor at the appropriate location based on time step.

This process will require the use of a `for` loop over the time dimension. After processing the entire target sequence, the output tensor should be returned. The cross entropy loss in `train.py` will then optimize model weights to increase the probability of the ground truth words.

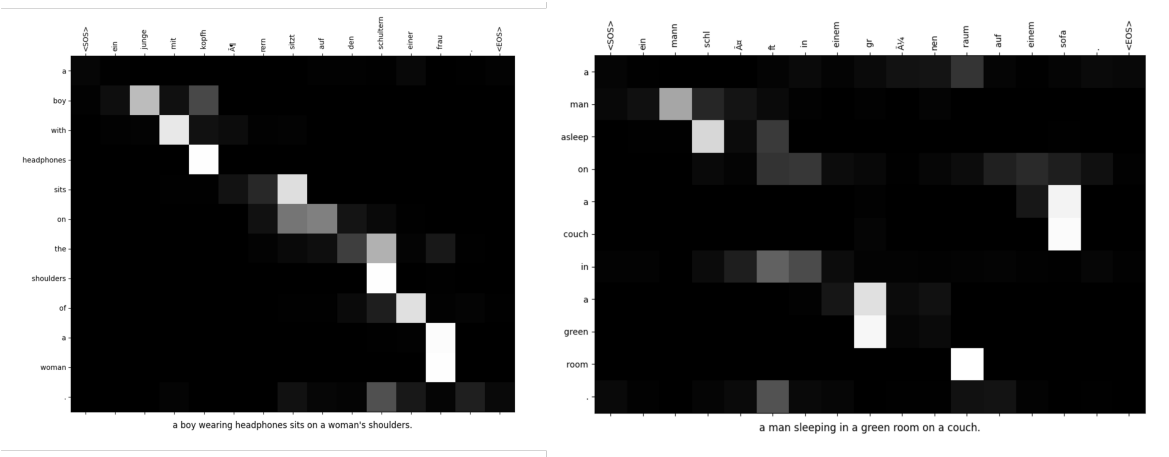
▶ TASK Q19 Implement Seq2Seq Translation [3pt]. Complete the Seq2Seq model by implementing the following function to enable autoregressive generation from our model. When translating a new sentence, we only have access to the source language and need to alter our logic from the `forward` function to enable autoregressive generation.

```
class Seq2Seq(nn.Module):

    . . .

    def translate(self, src, src_lens, sos_id=1, max_len=50):
        #tensors to store decoder output word IDs and attention matrices
        outputs = torch.zeros(src.shape[0], max_len).to(src.device)
        attns = torch.zeros(src.shape[0], max_len, src.shape[1]).to(src.device)

        # Tensor of initial <SOS> inputs
        input_words = torch.ones(src.shape[0], dtype=torch.long).to(src.device)
        input_words = input_words*sos_id

        #TODO

        return outputs, attns
```

This `translate` function differs from `forward` in two major ways: (1) rather than a ground truth `trg` word at each time step, the input words for the next time step should be those with the highest predicted scores from the model (i.e., argmax decoding), and (2) we would like to also return the attention matrices produced by the decoder for visualization. The skeleton code already provides `output` and `attns` tensors to store the generated word IDs and attention matrices. Further, we start the `input_words` off as a tensor of IDs corresponding to the <SOS> token. Autoregressive generation should continue for `max_len` time steps.

If everything is implemented correctly, you should be able to run `train.py` to build your model and execute

training – this should take roughly 30 minutes on a GPU. Logging to `wandb` has already been setup and correct models should be able to achieve per-word validation accuracies near 60-62%. Further, the training scripts also logs attention distribution visualizations for 10 examples from the validation set like those shown below.



Each shows the argmax predicted translation for a validation input sentence along with a visualization of the attention distributions. In each, the input German sentence is shown word-by-word along the top axis and the model-generated translation is shown word-by-word along the left axis. Each row shows the attention distribution (`alphas`) at that time step where brighter values are nearer to 1. The ground truth translation is shown below the figure. The top-right example is a particularly interesting case where the inversion of "on a couch" and "in a green room" between the two languages can be observed in the attention patterns. It is worth reflecting on the fact that the model was never given any explicit guidance about how these patterns should look, rather they emerge as part of minimizing prediction error for the English sentences.

▶ `TASK Q20 Train model and examine attention patterns [1pt].` Train your model with default hyperparameters and provide the training and validation plots in your report. Also provide and discuss two attention visualizations.

3.3 Experimentation

▶ `TASK Q21 Experiment with attention mechanisms [1pt].` Rerun your training experiments while setting the "`attn`" value in the configuration to "none" and "mean" to use no attention mechanism and one which averages the input encoders `word_representations` in place of the dot product attention. Provide your plots from `wandb` comparing these to our default attention-based model. Describe what you observe.

▶ `TASK Q22 Experiment with dropout [1pt].` With the default configuration, set the "`dropout`" parameter to 0 such that the Dropout layers have no effect during training or inference. Provide your plots from wandb comparing these to our default dropout probability of 0.5. Describe what you observe.

4 Debriefing (required in your report)

1. Approximately how many hours did you spend on this assignment?
2. Would you rate it as easy, moderate, or difficult?
3. Did you work on it mostly alone or did you discuss the problems with others?
4. How deeply do you feel you understand the material it covers (0%–100%)?
5. Any other comments?